

סנכרון

מערכות הפעלה
לינוקס

סנכרון

בפרק זה נדון בבעיות שונות שעלולות להתרחש כתוצאה מהרצה בו זמנית של מספר Thread-ים, בין אם הם שייכים לאותו Process או ל-Process-ים שונים. נסקור מנגנונים שונים שפותחו על מנת לפתור בעיות אלו, נלמד על מנגנוני הסנכרון בין Process-ים ו-Thread-ים, מנגנונים שקרויים Mutexes ו-Critical Sections. לבסוף נפתור בעיה קלאסית בתחום מדעי המחשב- בעיית הפילוסופים הסועדים.

סכרון

מהו Race Condition

המושג הראשון אותו נלמד הוא Race Condition. כדי להבין אותו, נבצע ניסוי קטן. לפנינו קוד, שפותח שני Thread'ים ומעביר לכך אחד מהם כתובת של אותו משתנה. כל אחד מה-Thread'ים מקדם את ערכו של המשתנה 10000 פעם. עיברו על הקוד הבא וענו על השאלה הבאה (בלי להריץ את הקוד): מה יהיה ערכו של המשתנה param בסיום הריצה?

```
#define NUM_THREADS 2
```

```
#define MAX 10000
```

```
VOID inc(PINT param) {  
    INT temp = *param;  
    temp++;  
    *param = temp;  
}
```

```
DWORD WINAPI thread_func(LPVOID param) {  
    for (INT i = 0; i < MAX; i++) {  
        inc((PINT)param);  
    }  
    return 1;  
}
```

```
int main()
{
    INT val = 0;
    LPVOID param = &val;
    HANDLE hThread[NUM_THREADS];
    for (INT i = 0; i < NUM_THREADS; i++) {
        hThread[i] = CreateThread(
            NULL,          //default security attributes
            0,             //default stack size
            thread_func,    //thread function
            param,          //thread param
            0,             //default creation flags
            NULL            //return thread identifier
        );
    }

    WaitForMultipleObjects(NUM_THREADS, hThread, TRUE, INFINITE);
    printf("Param value: %d\n", *(PINT)param);
}
```

באופן טבעי, היינו מצפים שבסיום הריצה ערכו של param יהיה שווה ל-20000. כעת הריצו את התוכנית מספר פעמים- מה קבלתם?

הנה תוצאות של ארבע הרצות- באף אחת מהן התוצאה אינה קרובה לתוצאה הצפויה.

Microsoft Visual Studio Debug Console
12595

Microsoft Visual Studio Debug Console
17531

Microsoft Visual Studio Debug Console
17004

Microsoft Visual Studio Debug Console
13480

לא רק שהתוצאות אינן 20000, יש גם שונות רבה מאד בין התוצאות. מחשבים לא אמורים לתת תוצאה שונה בכל פעם שמריצים את אותה תוכנה. מה מתרחש כאן?

לשני ה-Thread יש את אותה כתובת של param, כלומר שניהם קוראים וכותבים לאותה כתובת בזיכרון. הבעיות מתרחשות כאשר מתבצע Context Switch בזמן לא טוב. (תזכורת: Context Switch הוא ארוע שמתוזמן על ידי ה-Scheduler של מערכת ההפעלה, ובעקבותיו המעבד עובר מהרצה של קוד של Thread כלשהו להרצה של קוד של Thread אחר)

האיור הבא ממחיש מהו זמן לא טוב לביצוע Context Switch. שימו לב מתי הוא מתבצע- בדיוק אחרי ש- Thread1 העלה את ערכו של temp, ולפני שהוא העתיק את temp העדכני לתוך param.

Thread 1

```
INT temp = *param;  
temp++;
```

Thread2 מתחיל לפני
ש-Thread1 עידכן את
param

```
*param = temp;
```

Thread 2

```
INT temp = *param;  
temp++;  
*param = temp;
```

Thread1 משתמש בערך
לא מעודכן של temp

```
INT temp = *param;
```

```
temp++;
```

```
*param = temp;
```

ניקח דוגמה, בה param שווה 5. במקרה זה, Thread1 קורא את הערך מהזיכרון, ומחשב את temp להיות 6. ערכו של temp עדיין לא נשמר בחזרה לתוך param.

ברגע זה מתבצע Context Switch אל Thread2, שגם הוא קורא את param, שעדיין ערכו הוא 5. לכן Thread2 יעדכן את ערכו של param להיות 6.

כאשר Thread1 יחזור, הוא ישמור ב-param את temp. המשתנה temp הוא משתנה מקומי, כלומר ל-Thread1 יש משתנה temp שונה מאשר ל-Thread2. ערכו של temp שבתוך Thread1 הוא 6. לכן Thread1 ישמור 6 לתוך param.

התוצאה תהיה שלמרות ש-param התעדכן על ידי שני ה-Thread, מה שלכאורה היה צריך להעלות את ערכו ל-7, למעשה ערכו נותר 6.

התופעה שסקרנו כרגע נקראת Race Condition. ישנה "תחרות" בין שני ה-Thread מי יעדכן את ערכו של param, והתוצאה היא שהם מפריעים זה לזה לבצע את המשימה.

קטע קוד נקרא "קרייטי", או באנגלית Critical Section, אם אסור שבמהלכו יבוצע Context Switch. כפי שאנחנו רואים, קטע הקוד שסקרנו הוא קטע קוד קרייטי.

התבוננו בשורת קוד הבאה. שורת הקוד מעדכנת את ערכו של המשתנה ש-param מצביע עליו ועושה זאת בפקודה אחת בלבד. מה דעתכם, האם ביצוע העדכון באופן הזה ימנע את ה-Race Condition, כך שלא תתבצע החלפה בין Thread'ים בין פקודת ההעתקה לפקודת העדכון והשמירה חזרה של הערך העדכני?

```
*param += 1;
```

ובכן, התשובה היא לא, למרות שכעת פקודת העדכון נכתבת בשורה אחת הבעיה לא נפתרה.

כדי להבין את הסיבה לכך, נבדוק איך הפקודה הזו מתורגמת לקוד אסמבלי, קל לעשות זאת באמצעות טאב ה-Disassembler של Visual Studio, אשר מופיע בכל הרצה במצב Debug:

00DF18DF	mov	eax,dword ptr [param]
00DF18E2	mov	ecx,dword ptr [eax]
00DF18E4	add	ecx,1
00DF18E7	mov	edx,dword ptr [param]
00DF18EA	mov	dword ptr [edx],ecx

מסתבר כי הפקודה הבודדת שכתבנו בשפת C מתורגמת ל-5 פקודות אסמבלי שונות. העמודה השמאלית היא הכתובת בזיכרון שבה נמצאת כל פקודה, הכתובות עשויות להיות שונות מהכתובות בדוגמה. הפקודה הראשונה מעתיקה את ערכו של param לתוך רגיסטר eax, כך ש-eax שווה לכתובת ש-param מצביע עליה. הפקודה השנייה מעתיקה את הערך שנמצא בכתובת של eax לתוך רגיסטר אחר, ecx. כך נראה קוד אסמבלי של עבודה עם מצביעים. לאחר מכן מתבצע עדכון של ecx והשמה שלו באמצעות שתי פקודות נוספות אל הכתובת ש-param מצביע עליה.

כפי שאנחנו רואים, גם אם קוד כתוב בפקודה אחת בשפה עילית, אין הדבר אומר שהוא יתורגם לפקודה בודדת בשפת אסמבלי. לכן, עלול להתבצע Context Switch במהלך ריצה של פקודה בודדת בשפה עילית.

בתוכנית שלנו, ישנו קטע קוד קריטי:

```
INT temp = *param;  
temp++;  
*param = temp;
```

הפתרון הוא אובייקט שנקרא מנעול, Lock.

כזכור, אובייקט הוא מבנה נתונים שכולל משתנים ופונקציות שפועלות על המשתנים. לאובייקט מסוג Lock יוגדרו:

- משתנה "מצב מנעול" - 0 אם המנעול סגור, 1 אם המנעול פתוח
- פונקציות שבודקות את מצב המנעול
- פונקציות שמשנות את מצב המנעול

תרשים הזרימה הבא מתאר את אופן הפעולה של Lock. שימו לב, שאת השמות של הפונקציות שמבצעות את הבדיקות השונות, אנחנו עדיין לא מכירים. כרגע אנחנו רק מתארים באופן כללי איך Lock מופעל לצד קטע קוד קריטי.

פונקציה שנכיר בהמשך

בדוק אם המנעול פתוח

לא

כן

פונקציה שנכיר בהמשך

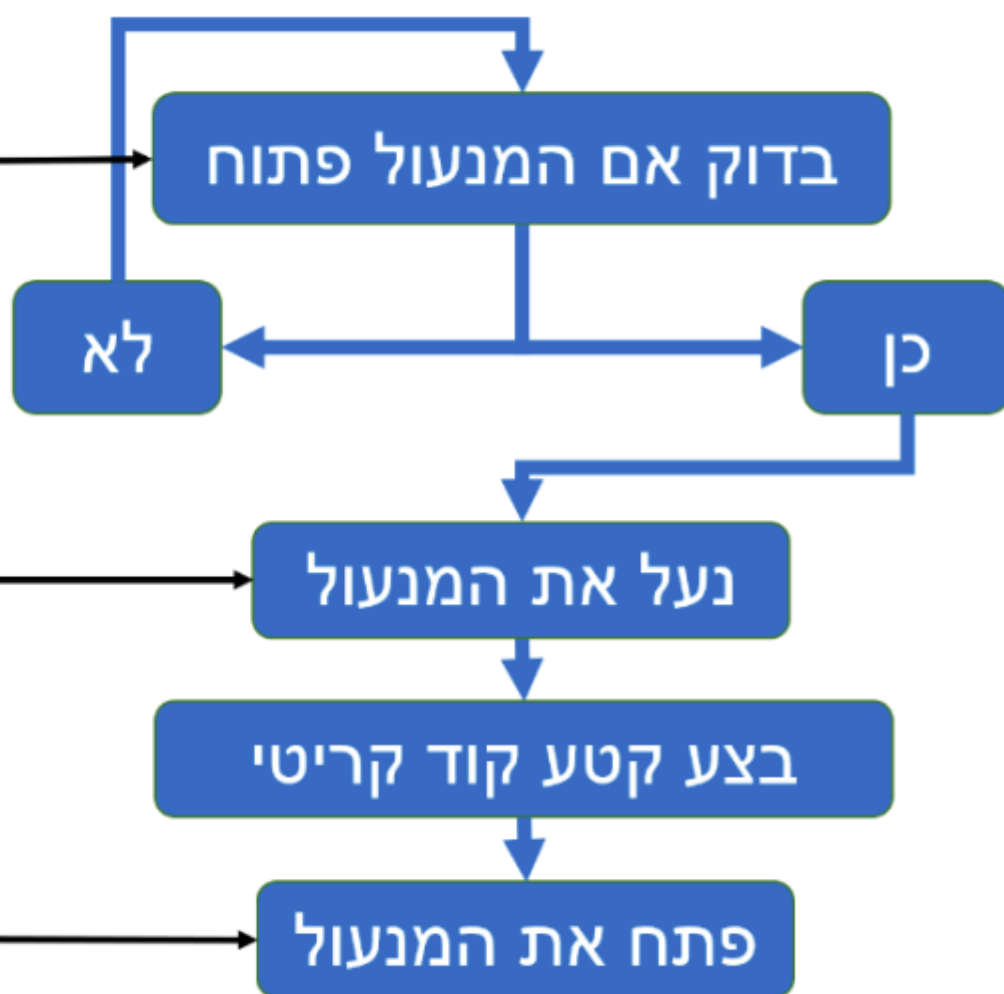
נעל את המנעול

```
INT temp = *param;  
temp++;  
*param = temp;
```

בצע קטע קוד קריטי

פונקציה שנכיר בהמשך

פתח את המנעול



Thread שמגיע לקטע קוד קריטי יצטרך קודם כל לבדוק אם המנעול פתוח. אם לא, הוא יצטרך להמתין לפתיחת המנעול. יכול מאד להיות שתוך כדי ההמתנה, יבוצע Context Switch ל-Thread אחר, שביצע את הנעילה.

אם המנעול פתוח, לפני הכל נועלים אותו. המטרה היא למנוע מ-Thread אחר לנעול את המנעול לפנינו. פעולת נעילת המנעול היא פקודה אחת בשפת מכונה, כך שלא יכול להיות שהמעבד יבצע Context Switch במהלכה. אם להשתמש במונח שלמדנו זה עתה, נעילת מנעול היא פעולה אטומית.

לאחר שהנעילה בוצעה, ניתן לגשת בשקט לביצוע קטע הקוד הקריטי. במקרה הכי גרוע, בו יבוצע Context Switch ל-Thread אחר, ה-Thread האחר ייתקל בנעילה ויישאר על המנעול עד שמשאבי המעבד יחזרו אל ה-Thread שנמצא בתוך קטע הקוד הקריטי.

לבסוף, עם סיום קטע הקוד הקריטי, ה-Thread משחרר את המנעול ובכך הוא מאפשר בנדיבות גם ל-Thread אחרים לבצע את קטע הקוד הקריטי.

האיור הבא ממחיש איך מנגנון ה-Lock פותר את הבעיה של Context Switch תוך כדי ביצוע קטע קוד קריטי.

Thread 1

```
INT temp = *param;  
temp++;
```

מבוצע context switch

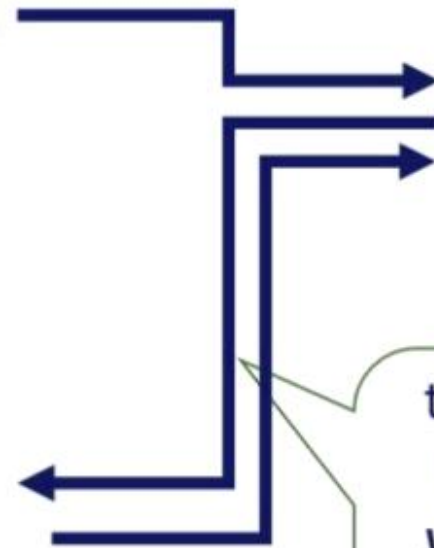
```
*param = temp;
```

Thread1 משלים את
העדכון ומשחרר את
הנעילה. מבוצע
context switch
אל thread2

Thread 2

```
INT temp = *param;  
temp++;  
*param = temp;
```

מנעול מונע מ-thread2
לגשת ל-param והוא
ממתין לפתיחה. מבוצע
context switch
אל thread1



כעת, לאחר שהבנו את התפקיד של מנעולים, נסקור שני סוגים של מנעולים: CriticalSection ו-Mutex. לכל סוג של מנעול יש שימוש משלו ופונקציות מיוחדות לו במערכת ההפעלה.

מנעול CriticalSection

המנעול CriticalSection משמש לסנכרון בין Threadים בתוך אותו Process. שימו לב, שהשם של המנעול זהה למושג ה-Critical Section, קטע (קוד) קריטי. לכן כדי להבחין ביניהם נקרא למנעול CriticalSection, ללא רווח בין המילים.

אנחנו צריכים ליצור אובייקט תוכנה כלשהו שכל ה-Threadים יכירו. לפני שתמשיכו לקרוא עיצרו וחישבו- איך הייתם מייצרים אובייקט שכל ה-Threadים יכירו?

אחד הדברים הבסיסיים שלמדנו על Threadים של אותו Process, הוא שיש להם זיכרון משותף. לכן, הפתרון קל: את מצב המנעול נשמור בתוך משתנה גלובלי. משתנה גלובלי מוכר על ידי כל ה-Threadים.

להלן הסבר השלבים של יצירת מנעול CriticalSection ושימוש בו.

1. נגדיר משתנה גלובלי מסוג CRITICAL_SECTION. ה-type שנקרא CRITICAL_SECTION מוגדר על ידי Windows ולכן כל מה שאנחנו נדרשים לעשות כדי שה-type יהיה מוכר זה לבצע include ל-Windows.h.

2. מתוך main, נקרא לפונקציה InitializeCriticalSection. כעת המנעול שלנו מוכן לשימוש.

3. כאשר Thread כלשהו רוצה לקחת בעלות על ה-CriticalSection, הוא יכול לעשות זאת רק אם המנעול אינו נעול. קיימות שתי פונקציות שמבצעות זאת, כל אחת שימושית למקרה אחר:

a. EnterCriticalSection - פונקציה זו ממתינה עד שהמנעול פתוח, ואז תופסת עליו בעלות. במילים אחרות, אם המנעול סגור, ה-Thread שלנו ימתין ויחכה, גם אם זה לנצח.

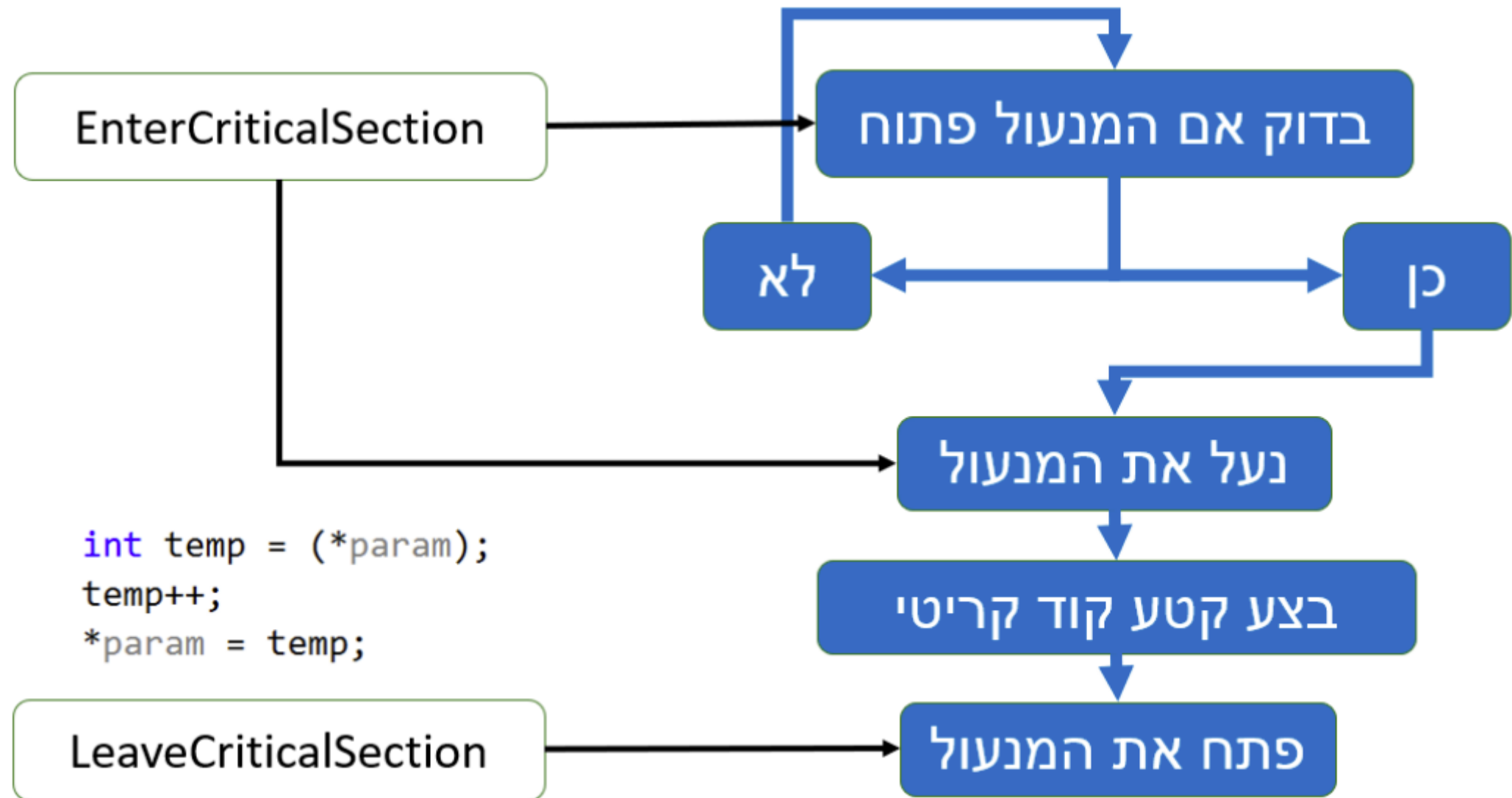
b. TryEnterCriticalSection – פונקציה זו תופסת בעלות על מנעול פתוח. בניגוד לפונקציה הקודמת, אם המנעול סגור הפונקציה לא ממתינה אלא מחזירה True / False. כלומר ה-Thread אינו ממתין למנעול והוא יכול להמשיך לבצע פעולות אחרות.

להלן הסבר השלבים של יצירת מנעול CriticalSection ושימוש בו.

4. כאשר ה-Thread סיים להשתמש במנעול, הוא יבצע LeaveCriticalSection, כדי לשחרר את המנעול ל-Thread אחרים שאולי זקוקים לו.

5. לאחר שכל ה-Thread ים סיימו את עבודתם (ראינו כבר שיש פונקציה שממתינה להם), פונקציית ה-main מוחקת את המנעול ומשחררת את המשאבים הקשורים אליו, באמצעות הפונקציה DeleteCriticalSection.

כעת כשאנחנו מכירים את הפונקציות הללו, אפשר לעדכן את תרשים הזרימה של מנעול מסוג CriticalSection, שמגן על קטע הקוד הקריטי שניתן כדוגמה מוקדם יותר:



להלן שלד של קוד שמבצע את שלבי העבודה עם critical section. זה אינו קוד שמתקפמל בצורה תקינה, אך כולל את המבנה הכללי שלתוכו צריך להוסיף את הקוד הקריטי:

```
CRITICAL_SECTION ghCriticalSection;
```

```
int main(void)
{
    InitializeCriticalSection(&ghCriticalSection)

    ...

    DeleteCriticalSection(&ghCriticalSection);
}
```

```
DWORD WINAPI ThreadProc(LPVOID lpParameter)
{
    EnterCriticalSection(&ghCriticalSection);

    // Place critical code here
    ...

    LeaveCriticalSection(&ghCriticalSection);

    return 1;
}
```

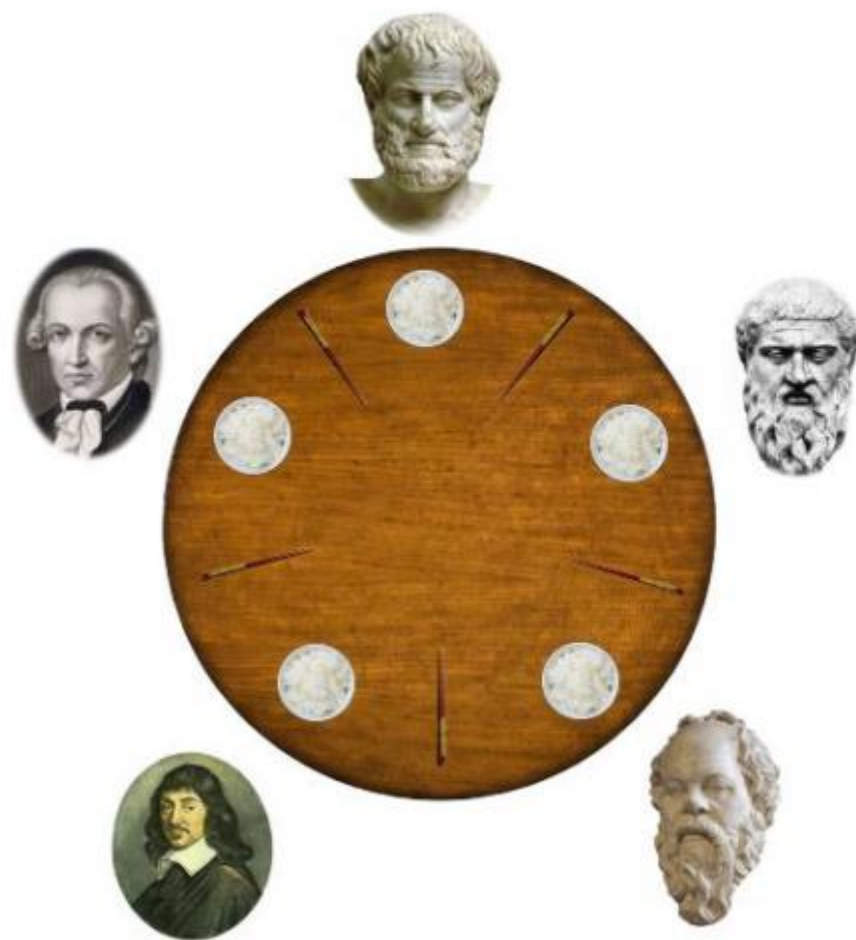
דמיינו צומת שבה יש אור ירוק בכל הכיוונים. איך תראה זרימת התנועה בצומת? סביר שכפי שנראית התנועה בתמונה הבאה- הרכבים חוסמים זה את זה כך שאף נתיב אינו מקבל זרימה חופשית.



תופעה דומה עלולה להתרחש גם בעולם המחשבים, בעקבות תכנון לא נכון. נניח שיש לנו שני Threadים ושני משאבים, וכל Thread מקבל פקודה לתפוס את שני המשאבים, לבצע פעולה מסויימת ואז לשחרר את המשאבים לרשות ה-Thread השני. הבעיה תיגרם כאשר Thread אחד יתפוס משאב אחד ואילו ה-Thread השני יתפוס את המשאב השני. בשלב זה כל אחד מה-Threadים ממתין לשחרור המשאב השני, ולכן גם אינו משחרר את המשאב שברשותו. אין פתרון למצב זה, הוא יכול להמשך לנצח. לכן התופעה נקראת- גם בעולם המחשבים וגם בעולם האמיתי- בשם Deadlock.

נדמיין שלצומת העמוס שלנו הגיע שוטר תנועה. כדי למנוע מצב של Deadlock, השוטר עוצר לחלוטין את זרימת התנועה משלושה כיוונים ומאפשר רק למכוניות שמגיעות לצומת מכיוון אחד להמשיך באופן חופשי. המפקד של שוטר התנועה שואל אותו בקשר מה הסטטוס, והשוטר עונה "התנועה זורמת". אכן, התנועה זורמת למי שהתמזל מזלו להגיע מהכיוון הנכון, אולם כל מי שהגיע לצומת משלושת הכיוונים החסומים יכול להמשיך לחכות... לתופעה זו קוראים Starvation, הרעבה. בניגוד ל-Deadlock, כאן יש מי שמקבל את המשאבים, אולם יש כאלה שלעולם לא יזכו לקבל אותם.

בעיית הפילוסופים הסועדים היא בעיה קלאסית מתחום מדעי המחשב. חמישה פילוסופים יושבים ליד שולחן, במרכז השולחן יש קערת אורז וליד כל פילוסוף יש מקל אכילה אחד בלבד. כמובן שכל סועד נזקק לשני מקלות אכילה כדי לאכול.



עליכם לכתוב תוכנה שתמצא דרך שבה כל פילוסוף יזכה לזמן אכילה. כאשר פילוסוף כלשהו אוכל, התוכנה תדפיס הודעה על מספר הפילוסוף שאוכל, בסיום האכילה הפילוסוף ישחרר את מקלות האכילה לסועד הבא. היזהרו מ-Deadlocks! אילו כל פילוסוף יחזיק במקל אכילה אחד בלבד, כל הפילוסופים יישארו רעבים. מאידך, היזהרו גם מ-Starvation. אילו שני פילוסופים יחזיקו בשני מקלות אכילה כל אחד, הם יאכלו לשובע אך כל יתר הפילוסופים ירעבו.

```

#include <windows.h>
#include <stdio.h>

#define PHILOSOPHERS 5
#define EAT_TIMES 3

CRITICAL_SECTION forks[PHILOSOPHERS];
HANDLE threads[PHILOSOPHERS];

DWORD WINAPI Philosopher(LPVOID param) {
    int id = (int)(SIZE_T)param;
    int leftFork = id;
    int rightFork = (id + 1) % PHILOSOPHERS;
    int eaten = 0;
    while (eaten < EAT_TIMES) {
        printf("Philosopher %d is thinking\n", id);
        Sleep(100);

        if (TryEnterCriticalSection(&forks[leftFork])) {
            if (TryEnterCriticalSection(&forks[rightFork]))
            {
                printf("Philosopher %d is eating (%d)\n", id, eaten + 1);
                Sleep(300);
                eaten++;
                LeaveCriticalSection(&forks[rightFork]);
            }
            LeaveCriticalSection(&forks[leftFork]);
        }
        Sleep(50);
    }
    printf("Philosopher %d finished eating\n", id);
    return 0;
}

```

```
int main() {  
    for (int i = 0; i < PHILOSOPHERS; i++) {  
        InitializeCriticalSection(&forks[i]);  
    }  
    for (int i = 0; i < PHILOSOPHERS; i++){  
        threads[i] = CreateThread(  
            NULL,  
            0,  
            Philosopher,  
            (LPVOID) (SIZE_T) i,  
            0,  
            NULL  
        );  
    }  
    WaitForMultipleObjects(PHILOSOPHERS, threads, TRUE, INFINITE);  
    for (int i = 0; i < PHILOSOPHERS; i++) {  
        CloseHandle(threads[i]);  
        DeleteCriticalSection(&forks[i]);  
    }  
    printf("All philosophers finished eating\n");  
    return 0;  
}
```

המנעול Mutex משמש לסנכרון בין Thread ים שנמצאים ב-Process ים שונים. כאשר דנו ב- Multi-Processing לעומת Multi-Threading, ראינו דוגמאות לכך שתוכנות פותחות Process ים רבים לטובת ביצוע המשימות שלהן.

לעיתים קרובות Process ים צריכים לתקשר זה עם זה, בפרט כאשר מדובר ב-Process ים שונים של אותה התוכנה. לדוגמה, Process ים שכרום פתח לטובת הרצת תוספים צריכים לתקשר עם Process ים שנוצרו לטובת טאבים שהמשתמש פתח בדפדפן. דוגמה נוספת, ה-Process שאחראי על הממשק הגרפי של ספוטיפיי צריך לתקשר עם ה-Process שאחראי על השמעת השירים, כדי להעביר פקודות כמו "עצור" ו"נגן".

לכן, הכרחי של-Process שונים יהיו דרכי תקשורת זה עם זה. אחת מדרכי התקשורת הנפוצות בין Processים היא שימוש במשאב משותף, כגון זיכרון. נלמד בהמשך אודות זיכרון משותף, כרגע כל מה שצריך לדעת הוא שאפשר לגרום לאזור זיכרון להיות "שייך" ליותר מ-Process אחד. כמובן, שפעולות הכתיבה והקריאה לזיכרון המשותף צריכות להיות מתואמות, אחרת עלולות להיווצר בעיות קשות, כפי שראינו כשסקרנו Race Conditions. איכשהו, כל Process צריך לקבל בעלות על הזיכרון המשותף בזמן שיתר ה-Processים לא יכולים לגשת אל הזיכרון המשותף. וכדי לבצע את הסינכרון הזה, CriticalSection כבר אינו מתאים. חישבו- מדוע?

התשובה היא, שמשתנים גלובליים אינם מוכרים מחוץ ל-Process שבו הם מוגדרים. כשמערכת ההפעלה מבצעת Context Switch בין Processים, כל המשתנים הגלובליים שהיו שייכים ל-Process הקודם פשוט אינם קיימים יותר מבחינת ה-Process החדש. אם כך, אנחנו צריכים למצוא פתרון אחר.

הפתרון הוא שימוש באובייקטים של מערכת ההפעלה. ב-Windows מוגדר טיפוס מסוג Object. כל

האובייקטים של Windows מנוהלים על ידי ה-Object Manager. אפשר לפנות אליו ולבקש Handle לאובייקט כלשהו, ואם ה-Object Manager ימצא שיש למבקש הרשאה מתאימה, הוא יעניק לו Handle לאובייקט המבוקש.

אם כך, המתכנת מגדיר אובייקט מסוג Mutex וקובע את השם שלו (בקרב נבצע זאת בעצמנו) ואילו מערכות ההפעלה שומרות את האובייקט בתוכן ומספקות Handle שמאפשר גישה אליו. אם כך, Mutex ו-CriticalSection דומים בכך ששניהם מבצעים את אותה פונקציונליות של סינכרון, אך בעוד ש-CriticalSection נמצא בזיכרון של Process כלשהו, Mutex הוא אובייקט של מערכת ההפעלה ומי שמשתמש בו מקבל רק Handle אליו.

השימוש ב-Mutex דומה למדי לשימוש ב-CriticalSection, אך שמות הפונקציות והשימוש בהם שונה מעט. להלן שלבי העבודה עם Mutex.

שלב א' – CreateMutex

יצירת Mutex מתבצעת על ידי CreateMutex. הפונקציה מקבלת שם קבוע, שיהיה השם של ה-Mutex. השם הזה יכול להיות כל שם שהמתכנת בוחר. כעת יש שתי אפשרויות. האפשרות הראשונה, אם ה-Mutex הזה אינו קיים, הוא נוצר. היכן הוא נוצר? כפי שאמרנו, כל ה-Mutexים הם משאבים של מערכת ההפעלה. זה הכרחי כדי להבטיח שה-Mutexים יהיו מוכרים על ידי Threadים של Processים שונים, שאינם חולקים זיכרון משותף. האפשרות השנייה, אם ה-Mutex הזה כבר קיים, הוא לא ייוצר שוב אלא רק יתקבל אליו Handle, שיאפשר לעשות עליו פעולות שונות כמו לבדוק מה המצב שלו והאם הוא תפוס.

```
HANDLE hMyMutex = CreateMutexA(  
    NULL,          // default security attributes  
    FALSE,         // initially not owned  
    "Hello");      // named mutex
```

שימו לב שהקוד הזה צריך להיות בשימוש גם ב-Process האב, שיוצר את ה-Mutex, וגם בכל ה-Processים הבנים, שצריכים להשיג אליו Handle.

שלב ב' - WaitForSingleObject

שלב זה מתרחש רק ב-Processים הבנים. כדי להשיג בעלות על ה-Mutex, משתמשים ב- WaitForSingleObject. הפונקציה הזו מקבלת את השם של ה-Mutex המבוקש ומנסה להשיג עליו בעלות. הפונקציה מקבלת גם פרק זמן מקסימלי כדי לנסות להשיג את ה-Mutex. אם פרק הזמן הוא INFINITE, אז הפונקציה תמתין כמה זמן שצריך עד שה-Mutex יהיה פנוי (בדומה ל-EnterCriticalSection, שמחכה כמה זמן שצריך ובינתיים התוכנית עוצרת).

לעומת זאת, אם פרק הזמן המקסימלי נקבע ל-0, אז במידה וה-Mutex אינו זמין הפונקציה תמשיך הלאה בלי המתנה (בדומה ל-TryEnterCriticalSection).

```
DWORD waitResult = WaitForSingleObject(  
    hMyMutex,  
    INFINITE); // no time-out interval
```

```
DWORD waitResult = WaitForSingleObject(  
    hMyMutex,  
    0);          // zero time-out interval
```

בנוסף על הבדיקה האם ה-Mutex פנוי, הפונקציה גם תופסת אותו אם הוא פנוי, לכן אין צורך להשתמש בפונקציה נוספת לתפיסת ה-Mutex.

הפונקציה מחזירה אחד מתוך ארבעה ערכים המוגדרים בתיעוד ב-MSDN (חפשו בגוגל msdn waitforsingleobject, היכנסו לתיעוד וקיראו עליהם). הערך שמעניין אותנו הוא WAIT_OBJECT_0 שהמשמעות שלו היא שה-Mutex היה פנוי וכעת הוא ברשותנו.

קוד דוגמה לבדיקת ערך החזרה:

```
if (waitResult != WAIT_OBJECT_0) {  
    // Mutex not free  
    Bad();  
}  
else {  
    // Mutex free  
    Good();  
}
```

שלב ג' – ReleaseMutex

Process שסיים את השימוש ב-Mutex ומבקש לשחרר אותו לשימוש Process אחרים, משתמש ב-ReleaseMutex. זוהי פונקציה פשוטה ביותר שמקבלת רק Handle ל-Mutex המבוקש.

דוגמת קוד:

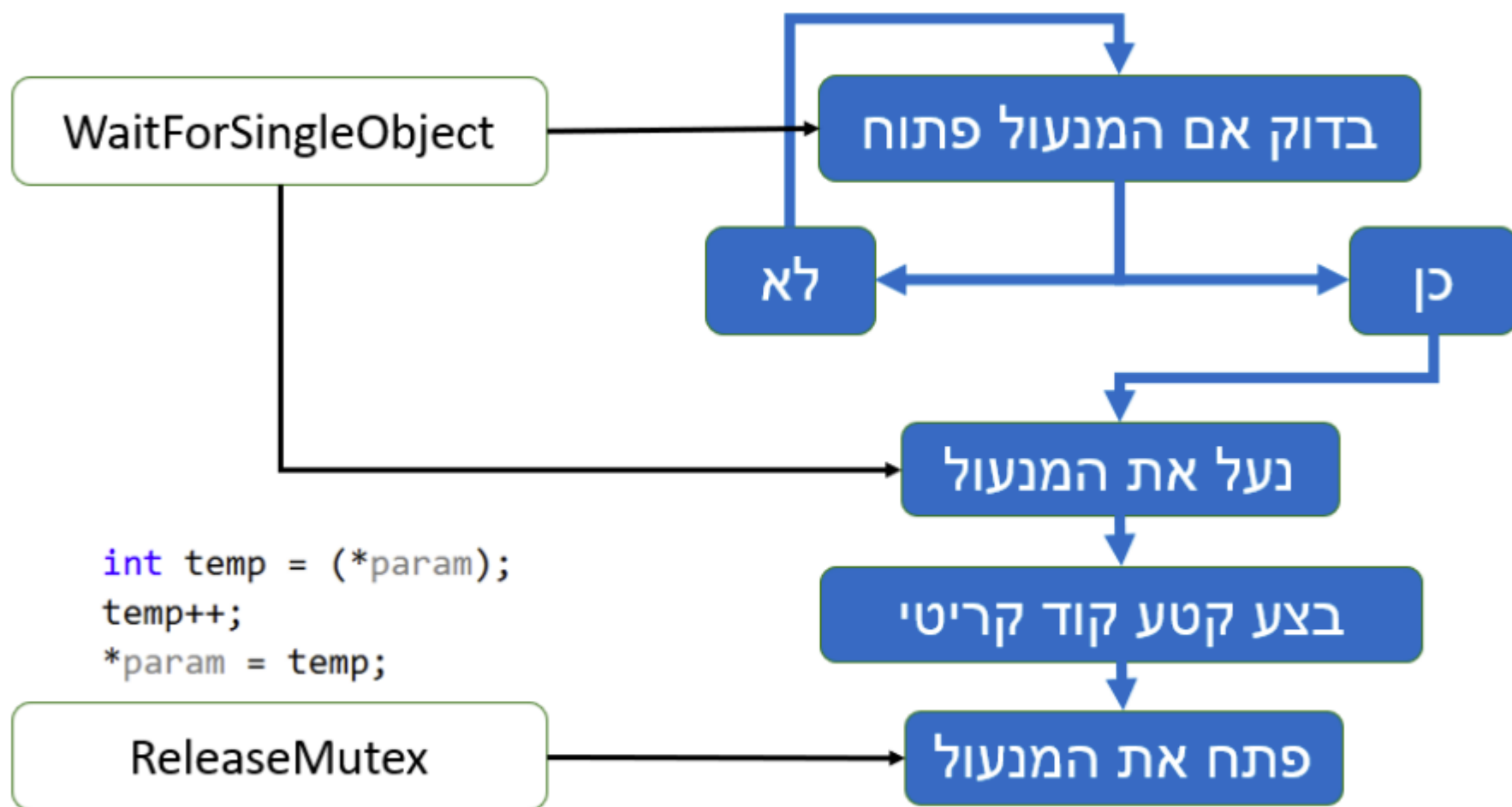
```
ReleaseMutex(hMyMutex);
```

שלב ד' – CloseHandle

בסיום השימוש ב-Mutex מומלץ לסגור את ה-Handle שעושה בו שימוש

```
CloseHandle(hMyMutex);
```

כעת כשאנחנו מכירים את הפונקציות הללו, אפשר לעדכן את תרשימים הזרימה גם של מנעול מסוג Mutex, שמגן על קטע הקוד הקריטי שניתן כדוגמה מוקדם יותר:



ניצור תוכנה בשם HelloWorldMutex שמדפיסה את המספר שהיא קיבלה כארגומנט משורת הפקודה. אבל, התוכנה לא מדפיסה מיד את המספר, אלא מחכה בנימוס כדי לוודא ששום תוכנה אחרת לא מבצעת את אותה פעולה כרגע.

ניצור גם תוכנה בשם MutexParentProcess. תוכנה זו תיצור שני Process שיריצו במקביל את HelloWorldMutex. אנחנו רוצים לראות הדפסה "מנומסת" שבה Process מדפיס רק אם הוא היחיד שמדפיס כרגע, ואם לא הוא ממתין.

הרצה תקינה של MutexParentProcess תוצג למסך באופן הבא:

Hello World! Process got argument 0

(המתנה של 5 שניות)

Hello World! Process got argument 1

(המתנה של 5 שניות)

סיום ריצה

כבסיס לקוד של HelloWorldMutex נשתמש בקוד של HelloWorld שפיתחנו בפרק הקודם, התוכנה שמדפיסה למסך את הפרמטר שהיא מקבלת מה-Process האב.

אנחנו צריכים להוסיף בפתיחה CreateMutex, כדי לקבל Handle. לאחר מכן נבצע בדיקה פשוטה אם ה-Mutex פנוי. אם כן- נבצע הדפסה ונשחרר את ה-Mutex.

```
#include "pch.h"
```

```
int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Incorrect number of arguments\n");
        return 0;
    }
    int id = atoi(argv[1]);
    HANDLE hMyMutex = CreateMutexA(NULL, FALSE, "Hello");
    DWORD waitResult = WaitForSingleObject(hMyMutex, 10000);
    if (waitResult == WAIT_OBJECT_0)
    {
        printf("Hello World! Process got parameter %d\n", id);
        Sleep(5000);
        ReleaseMutex(hMyMutex);
    }
    else
        printf("Mutex could not be obtained for too long\n");
    CloseHandle(hMyMutex);
    return 1;
}
```

איך צריך להראות הקוד של MutexParentProcess?

בפרק הקודם ראינו איך נראה קוד שיוצר Process ומעביר אליו פרמטר. השדרוגים שאנחנו צריכים לבצע הם שניים:

- להגדיר Mutex

- ליצור שני Processים, לא אחד כמו בפרק הקודם

ההגדרה של Mutex היא בדיוק כפי שראינו בדוגמת הקוד למעלה. כדי ליצור שני Processים פשוט ניצור לולאה שבכל איטרציה תיצור מחרוזת חדשה שתועבר כשורת פקודה. בנוסף, הפרמטר `pi` שאנחנו מעבירים ל-`CreateProcess` צריך להפוך למערך, כדי שתהיה לנו גישה ל-`Handle`ים של כל ה-`Process`ים שיצרנו. אנחנו חייבים לשמור גישה ל-`Handle`ים כדי לנוכל לקרוא ל-`WaitForSingleObject` ולהבטיח שריצת ה-`Process`ים הבנים הסתיימה לפני שנסיים ריצה.

לכן הקוד לאחר השינוי נראה כך:

```
#include "pch.h"
#define EXE_FILENAME    "..\\HelloWorldMutex.exe"
#define NUM_PROCESSES  2
```

```
int main()
```

```
{
```

```
    HANDLE hMyMutex = CreateMutexA(NULL, FALSE, "Hello");
```

```
    INT size = strlen(EXE_FILENAME) + 3; // length is increased by 3:
```

```
        // space character- 1 byte,
```

```
        // one digit number- 1 byte,
```

```
        // string NULL terminator- 1 byte
```

```
    PCHAR param = (PCHAR)malloc(size*sizeof(CHAR));
```

```
    STARTUPINFOA si;
```

```
    PROCESS_INFORMATION pi[NUM_PROCESSES];
```

```
    for (int i = 0; i < NUM_PROCESSES; i++) {
```

```
        sprintf_s(param, size, "%s %d", EXE_FILENAME, i);
```

```
        ZeroMemory(&si, sizeof(si));
```

```
        si.cb = sizeof(si);
```

```
        ZeroMemory(&pi[i], sizeof(pi[i]));
```

```
        CreateProcessA(
```

```
            NULL,
```

```
            param,
```

```
            NULL,
```

```
            NULL,
```

```
            FALSE,
```

```
            0,
```

```
            NULL,
```

```
            NULL,
```

```
            &si,
```

```
            &pi[i]);
```

```
    }
```

```
// Wait for every process to finish, close everything before return
for (int i = 0; i < NUM_PROCESSES; i++) {
    WaitForSingleObject(pi[i].hProcess, INFINITE);
}

ReleaseMutex(hMyMutex);

for (int i = 0; i < NUM_PROCESSES; i++)
{
    CloseHandle(pi[i].hProcess);
    CloseHandle(pi[i].hThread);
}

return 0;
}
```

עד עכשיו דנו בשני סוגים של מנעולים, שהיו בינאריים במהותם. כלומר, אם ניקח מנעול מסוג Mutex או מסוג CriticalSection, יש בכל רגע נתון רק Thread אחד שמחזיק במנעול.

המנעולים הבינאריים שלנו עבדו יפה כל עוד היה הגיון בכך שרק Thread יחיד יהיה הבעלים של המנעול, כי רצינו למנוע מ-Thread'ים אחרים גישה לקטע קוד קריטי. אך לפעמים המקרה הוא שונה. נשתמש בדוגמה מהחיים שבוודאי מוכרת לכולם- שירותים ציבוריים. השירותים מכילים מספר תאים. בכל פעם שנכנס אדם הוא תופס תא וכמות התאים הפנויים יורדת באחד, בכל פעם שאדם מסיים התא שלו מתפנה וכמות התאים הפנויים עולה באחד. אם הגעתם לשירותים שבהם כמות התאים הפנויים היא אפס, אתם נאלצים להמתין לתא פנוי.

באופן דומה גם במחשב יש לפעמים משאבים שיש להם מאפיינים של "שירותים ציבוריים". נניח לדוגמה שאנחנו רוצים להגביל גישה לקובץ כלשהו לכמות משתמשים מקסימלית, או להגביל את כמות החלונות שתוכנה פותחת. לשם כך נשתמש ב-Semaphore.

ה-Semaphore אינו מנעול אלא מנגנון איתות (באנגלית Signalling), שמאפשר ל-Thread לאותת ל-Thread אחרים כאשר אירוע מוגדר מתרחש, אך ניתן להשתמש בו באופן דומה למנעול. Semaphore מאפשר ליותר מ-Thread אחד להיות בעלים של משאב משותף. הדרך שבה הדברים מתבצעים היא שבזמן יצירת ה-Semaphore מוגדר מספר מקסימלי כלשהו. תפיסת משאב מורידה ב-1 את ערכו של ה-Semaphore, שחרור משאב מעלה את ערכו ב-1. אם הערך של ה-Semaphore שווה לאפס, לא ניתן לתפוס בעלות על המשאב. תכונה חשובה של Semaphore היא שהעלאת או הורדת הערך היא פעולה אטומית, כלומר פעולה שבמהלכה לא יכול להתבצע Context Switch, שאחרת הספירה היתה עלולה להיות שגויה.

לא נתעכב על הפונקציות השונות שמשמשות ליצירת ושימוש ב-Semaphore, הן די דומות לפונקציות שלמדנו עבור Critical Section ו-Mutex, וניתן למצוא אותן באתר MSDN בלינק

<https://docs.microsoft.com/en-us/windows/win32/sync/using-semaphore-objects>

ב-Windows קיימים שני סוגים של Semaphore. בתהליך יצירת ה-Semaphore נבחר אם לתת לו שם או לא. אם לא ניתן לו שם, הוא יהיה אובייקט מקומי של ה-Process ורק Thread ים באותו ה-Process יוכלו להשתמש בו, בדומה ל-CriticalSection. אם ניתן לו שם, מה שנקרא "Named Semaphore", הוא יהיה אובייקט של מערכת ההפעלה. לכן גם Thread ים מ-Process ים אחרים יכולים לקבל Handle ל-Named Semaphore. הפעולה מתבצעת באמצעות הפונקציה OpenSemaphore, שמקבלת את שם ה-Semaphore בתור פרמטר. כפי שראינו עם Mutex ים, כאשר Semaphore הוא אובייקט של מערכת ההפעלה יש לכך משמעות, כל קריאה אליו צריכה לעבור דרך מערכת ההפעלה. לכן הוא יותר איטי מאשר CriticalSection.

לסיכום, להלן טבלת השוואה בין שלושת המנגנונים שהכרנו:

Named Semaphore	Mutex	CriticalSection	
V	V (אפשרי אך פחות יעיל מאשר (Critical Section	V	סנכרון Thread ים שנמצאים באותו Process
V	V	X	סנכרון Thread ים ש*אינם* נמצאים באותו Process
מוגדר על ידי המשתמש	1	1	כמות גישות מקסימליות בו זמנית למשאב

מהתבוננות בטבלה נראה כאילו Mutex הוא מקרה פרטי של Named Semaphore, שבו המשתמש מגדיר את כמות גישות מקסימלית להיות 1. מקרה זה נקרא Binary Semaphore ואכן, מבחינה מעשית יש דמיון רב ל-Mutex. עם זאת, כאמור Semaphore הוא מנגנון איתות ולא מנעול. לכן אין Thread שהוא הבעלים שלו וכל Thread יכול להעלות או להוריד את הערך שלו, בניגוד ל-Mutex.